README

Documentación técnica · Proyecto Integrador Módulo 1 · Análisis de clientes de restaurantes en Estados Unidos · Juan Pablo Luna Moreno · Data Science, Henry · Octubre 2026

1. Descripción

El proyecto limpia y explora una base de 30.000 clientes de restaurantes, la complementa con datos de la API de Yelp y construye los análisis y la segmentación que sustentan las recomendaciones de negocio. Este documento explica **cómo está organizado el código, qué hace cada bloque, qué métodos se usaron y por qué se tomó cada decisión**. Los hallazgos principales están en la sección 2; el detalle de los resultados y las recomendaciones para el negocio está en **Recomendaciones.pdf**.

2. Hallazgos principales

- **El estrato socioeconómico define el gasto.** De él dependen las visitas al mes, el gasto por comida y la membresía. Entre quienes comen fuera, un cliente de estrato Bajo visita unas 2 veces al mes y gasta 12 USD por comida; uno de estrato Muy Alto, 7,5 veces y 64 USD.
- **Ingresos y gasto:** la correlación es de 0,71 en toda la base, pero dentro de cada estrato es cercana a 0. La relación existe solo entre estratos.
- **Ciudades:** Miami tiene el mayor gasto mensual (unos 226 USD por cliente) y el mayor porcentaje de membresías (53,2%). También es la ciudad con más clientes de estrato Alto y Muy Alto (61,5%).
- **Clientes de mayor gasto:** el 95,8% del top 10% es de estrato Muy Alto, y sus preferencias de comida son casi iguales a las del resto.
- **Alcohol:** el 90,3% de los clientes de 18 a 29 años consume licor, frente a un 53–56% en los demás rangos de edad.
- **Segmentación:** los segmentos Premium y Alto valor son el 48% de los clientes y generan el 83% del gasto.
- **Chicago vs Yelp:** Carnes es la comida más pedida (29,5% de los clientes) y, en Yelp, la peor calificada (4,0) y la más cara (2,9 de 4).

3. Estructura del proyecto

El repositorio **ProyectoM1_JuanPabloLuna** tiene dos carpetas: **Avances/**, con los notebooks y el CSV inicial, y **Documentacion/**, con los PDF.

Archivo	Tipo	Contenido
Avance_EDA.ipynb	Notebook	Avance 1 (Bloques 1–3): carga, limpieza, filtrado y tabulación de Chicago. Avance 3 (Bloques 4–7): análisis de todas las ciudades, gráficos, comparación de Chicago con Yelp y segmentación.
Avance_API_Yelp.ipynb	Notebook	Avance 2 (Bloques 1–3): conexión a la API de Yelp, extracción e integración con la base de clientes.
base_datos_restaurantes_USA_v2.csv	Entrada	Base original: 30.000 filas y 17 columnas. Es el único CSV que se sube al repositorio.

Archivo	Tipo	Contenido
clientes_chicago_limpio.csv	Generado	Clientes de Chicago ya limpios (Avance 1). Lo lee el Avance 2.
restaurantes_yelp_chicago.csv	Generado	1.000 registros de Yelp (680 restaurantes distintos). Se guarda para no repetir llamadas a la API.
integrado_clientes_yelp_chicago.csv	Generado	Tabla de demanda (clientes) contra oferta (Yelp) por preferencia alimenticia. La usa el Bloque 6 de Avance_EDA.
Documentacion/	Documentos	README.pdf (este documento) y Recomendaciones.pdf.

Los archivos de tipo **Generado** no se suben: se crean en la carpeta Avances al correr los notebooks.

4. Requisitos

- Python 3 y Jupyter Notebook.
- **pandas** y **numpy** en todos los avances. **requests** y **time** (este último viene con Python) en el Avance 2. **matplotlib** desde el Bloque 4 (en el Avance 1 los gráficos usan el .plot() de pandas). **os** (viene con Python) en el Bloque 6, para revisar si ya existe el archivo de Yelp.
- Conexión a internet y la API Key de Yelp entregada en clase, solo para los Bloques 1 y 2 del notebook de Yelp.

5. Cómo ejecutar

- Clonar el repositorio y trabajar dentro de la carpeta **Avances**: los notebooks y el CSV inicial están juntos porque las rutas son relativas.
- **Primero** correr Avance_EDA.ipynb (Kernel → Restart & Run All). El Bloque 3 genera clientes_chicago_limpio.csv. La primera vez, al final del Bloque 6 aparece el aviso “Falta el archivo integrado_clientes_yelp_chicago.csv”: es normal, porque ese archivo lo crea el notebook de Yelp.
- **Después** correr Avance_API_Yelp.ipynb (Restart & Run All). Lee clientes_chicago_limpio.csv y genera restaurantes_yelp_chicago.csv e integrado_clientes_yelp_chicago.csv.
- **Por último** volver a correr Avance_EDA.ipynb. Ahora el Bloque 6 muestra la comparación de Chicago con Yelp.
- Si una celda de la API queda cargando mucho tiempo, reiniciar el kernel antes de volver a correr. Una llamada colgada deja en cola todas las celdas siguientes.
- Si se acaban las llamadas del día (código 429), hay que esperar al día siguiente, porque la API Key es compartida por el curso. Las salidas guardadas en los notebooks muestran los resultados de la ejecución original.

6. Flujo de datos

Paso	Entrada	Proceso	Salida
1	base_datos_restaurantes_USA_v2.csv	Avance_EDA · Bloques 1–3	df_total (base completa limpia), df (Chicago) y clientes_chicago_limpio.csv
2	API de Yelp	Avance_API_Yelp · Bloques 1–2	restaurantes_yelp_chicago.csv
3	clientes_chicago_limpio.csv y restaurantes_yelp_chicago.csv	Avance_API_Yelp · Bloque 3	integrado_clientes_yelp_chicago.csv
4	df_total	Avance_EDA · Bloques 4–7	Tablas, gráficos y segmentación
5	integrado_clientes_yelp_chicago.csv	Avance_EDA · Bloque 6	Comparación de Chicago con Yelp (tabla y gráfico)

7. Qué hace cada bloque

Avance_EDA.ipynb

Bloque	Qué hace	Métodos principales
1. Carga	Lee el CSV, muestra el tamaño, las primeras filas, los tipos de dato y el resumen estadístico. Ahí ya se ven valores imposibles (edad −5 y 300, frecuencia −3).	read_csv, shape, head, info, describe
2. Limpieza	Detecta nulos, duplicados y errores en la base completa, los corrige una sola vez (df_total), filtra Chicago (df) y verifica que la limpieza quedó bien.	isnull().sum, duplicated, filtros booleanos, .loc, np.nan, fillna, astype, value_counts, copy
3. Tabulación	Tablas de frecuencia de cada variable, tablas cruzadas en %, gasto por comida por estrato, 4 gráficos iniciales. Guarda la base limpia de Chicago.	value_counts, crosstab(normalize), groupby + mean, reindex, DataFrame.plot, to_csv
4. Preparación	Crea gasto_mensual, gráfico de clientes por ciudad y listado por estrato.	operaciones entre columnas, describe, plot(kind='bar'), bar_label
5. Gasto	Gasto mensual por ciudad (y dentro de cada estrato), frecuencia contra gasto por comida, dispersión ingresos contra gasto y correlaciones.	groupby, pivot_table, plot de líneas, scatter, corr, agg(min, max)
6. Perfil	Preferencias por ciudad, top 10% de gasto, membresías por ciudad en cantidad y %, % de cada estrato por ciudad, consumo de licor por rango de edad y comparación de las preferencias de Chicago con la oferta en Yelp (si el archivo de Yelp ya existe).	crosstab, quantile(0.9), np.where, pd.cut, barras apiladas, os.path.exists, read_csv, bar
7. Segmentación	Crea 5 segmentos, calcula su perfil y su peso en clientes y gasto, y compara el gasto con y sin membresía dentro de cada estrato.	np.select, groupby + agg con nombres, pivot_table

Avance_API_Yelp.ipynb

Bloque	Qué hace	Métodos principales
1. Conexión	Arma la URL y el header con la API Key (Bearer), hace una consulta de prueba y revisa el código de respuesta (200, 401, 429) antes de leer los datos.	requests.get(params, headers), status_code, json()
2. Extracción	Por cada preferencia busca un término en inglés (steakhouse, seafood, fish, vegetarian, vegan) solo en restaurantes. Pide 4 páginas de 50 resultados con offset y hace pausas de medio segundo. Guarda nombre, categorías, calificación, reseñas, precio, dirección, código postal y coordenadas. Luego quita repetidos y convierte el precio a número.	bucles for, offset/limit, time.sleep, dict.get, join, DataFrame, drop_duplicates, str.len, to_csv
3. Integración	Lee los dos CSV, arma la tabla de demanda (clientes y gasto por preferencia) y la de oferta (calificación, reseñas y precio en Yelp), las une por preferencia alimenticia y guarda la tabla integrada que usa el Bloque 6 de Avance_EDA.	read_csv, groupby + agg, merge(how='left'), sort_values, to_csv

8. Decisiones de limpieza

Problema	Cantidad (base completa)	Tratamiento	Por qué
Edades imposibles (−5 y 300)	207	Se dejan vacías (np.nan)	No se conoce la edad real; rellenarlas sería inventar un dato.
Frecuencias de visita negativas (−3)	1.547	Se dejan vacías	Esos clientes sí tienen gasto, o sea sí visitan, pero no se sabe cuántas veces.
Preferencia alimenticia vacía	1.403	“No especificado”	Así esos clientes quedan como un grupo propio y no se pierden en las tablas.
Edad y gasto por comida vacíos de origen	101 y 145	Se dejan vacíos	Son menos del 1%; pandas los excluye de promedios y conteos.
Teléfono y correo vacíos	~50%	Sin cambios	No se usan en el análisis.
Filas o IDs duplicados	0	No aplica	No hay duplicados.
id_persona numérico	—	Se pasa a texto	Es un identificador, no un valor para sumar o promediar.
Yelp: calificación 0 con 0 reseñas	1	Se deja vacía	No es una mala nota; el restaurante no tiene calificación.
Yelp: precio vacío	242	Se deja vacío	Yelp no informa el precio de todos los restaurantes; el promedio se calcula con los que sí lo tienen.

Criterio de filtrado: el Avance 1 y la integración con Yelp se hacen sobre **Chicago** (5.384 clientes), la ciudad con más clientes, para trabajar con la muestra más grande. El Avance 3 usa todas las ciudades porque sus preguntas comparan ciudades.

9. Variables creadas

Variable	Cálculo	Para qué se usa
gasto_mensual	frecuencia_visita × promedio_gasto_comida	Comparar gasto entre ciudades, estratos y segmentos.
grupo_gasto	“Top 10%” si gasto_mensual ≥ percentil 90; si no, “Resto”	Perfilar a los clientes de mayor gasto.
rango_edad	pd.cut en 18–29, 30–44, 45–59 y 60–80	Comparar el consumo de licor por edad.
segmento	np.select según estrato y visitas	Segmentación final (5 grupos).
nivel_precio	Cantidad de símbolos $ del precio de Yelp (1 a 4)	Promediar el precio de los restaurantes.

10. Técnicas estadísticas

- **Correlación de Pearson** (corr) entre ingresos y gasto mensual, primero para todos los clientes y después dentro de cada estrato. Así se puede ver si la relación existe dentro de los grupos o solo entre ellos.
- **Percentil 90** (quantile) para definir objetivamente el top 10% de gasto.
- **Tablas cruzadas normalizadas** (crosstab con normalize) para comparar grupos de distinto tamaño en porcentaje.
- **Comparación dentro de grupos** (pivot_table por estrato y membresía) para no confundir el efecto de la membresía con el del estrato.
- **Promedios excluyendo ceros** en el gasto por comida: los clientes con 0 visitas tienen gasto 0 y bajarían artificialmente el promedio “por comida”.

11. Supuestos y limitaciones técnicas

- Se asume que frecuencia_visita está en visitas al mes. La base no lo dice, pero la consigna habla de gasto mensual.
- La base no tiene fechas, así que no hay análisis en el tiempo.
- Yelp entrega como máximo 50 resultados por llamada; se tomaron los primeros 200 por búsqueda. La búsqueda por palabra incluye restaurantes que solo mencionan el término, por eso no se usó el total que reporta Yelp.
- Los datos de Yelp cambian con el tiempo: si se vuelve a correr el Bloque 2, las cifras pueden variar un poco.
- La API Key es compartida por el curso y tiene un límite diario de llamadas.

12. Buenas prácticas aplicadas

- La limpieza se hace una sola vez sobre la base completa y se reutiliza en el Avance 1 y en el Avance 3.
- Se usa .copy() al filtrar para no modificar la tabla original.
- Los resultados de la API se guardan en CSV para no repetir llamadas.
- Se revisa el código de respuesta antes de leer los datos de la API y se hacen pausas entre llamadas.

- Antes de leer el archivo de Yelp, el Bloque 6 revisa que exista (os.path.exists). Así Avance_EDA corre completo sin errores aunque todavía no se haya corrido el notebook de Yelp.
- Cada paso del código tiene un comentario que explica qué hace y, cuando hay una decisión, por qué (“POR QUÉ:”).